

SWARN

Autonomous ML Engineering Platform

Design & Engineering Document

Version 3.0 · July 2026

Audience: Engineering · Confidential

Contents

1. Objective

Build and ship an autonomous ML engineering platform that outperforms **HeyNeo (heyneo.com)** — the current market leader claiming #1 on MLE-Bench (34.2%, Aug 2025) — on both benchmark performance and product capability, and take it to market.

ML engineering today is dominated by manual, slow iteration: writing training scripts, running experiments, reading logs, fixing failures, tracking metrics, and repeating for days. General coding agents (Cursor, Claude Code, Codex) write code but do not own the experiment loop. HeyNeo's core insight — an agent that runs the full engineering loop autonomously — is correct; our objective is to execute that insight better.

Success criteria

- **Benchmark:** match or beat HeyNeo-class results on MLE-Bench-style tasks through systematic tree search, parallel exploration, and cross-run learning.
- **Capability:** cover HeyNeo's entire documented surface (IDE/MCP integration, evals, fine-tuning, RAG, deployment) and exceed it where they are weak (transparent budgets, resumable runs, self-improvement, a self-hosted deployed model with no per-token API costs).
- **Trust:** every run produces inspectable artifacts — solution tree, code, metrics, reports — with no opaque credit system and no data leaving the user's infrastructure.
- **Time-to-market:** engine already implemented and tested (57/57 automated tests); productization gaps are enumerated in Section 11.

2. The Solution We Are Providing

Swarn is a fully autonomous ML engineer. Given a natural-language task ('Predict churn, maximize AUC') and a pointer to data, it plans, writes complete solution scripts, executes them in a sandbox, reviews the results, and iterates — debugging failures and improving winners — until a budget is spent. It then hands back the best solution, a full experiment report, and everything it learned.

Four properties distinguish it from both general coding agents and HeyNeo:

- **Systematic search, not a chat loop.** An AIDE-style solution tree (draft → debug → improve) with a reservation-aware parallel scheduler. Multiple complete solutions are generated, executed, and measured concurrently — HeyNeo expands one node at a time.
- **Self-improvement across runs.** After every run, a reflection step distills generalizable lessons into a bounded playbook, and the run is archived in a full-text-searchable index. Future runs receive the playbook and similar past runs as prior art. HeyNeo starts every task cold; Swarn compounds.
- **Radical transparency.** Crash-safe journals, resumable runs, explicit token/time/step budgets, per-run reports with the full solution tree, and a live dashboard. No opaque credits.
- **Runs anywhere, on your terms.** One self-hosted deployed model behind a single OpenAI-compatible endpoint (TEST: Qwen 3.5 9B on Modal; PRODUCTION: one config swap in agent/llm/router.py or SWARN_DEPLOYED_* env vars), Docker or plain subprocess execution, local-first data handling, and an MCP server that plugs into Claude Code, Cursor, Windsurf, and Zed.

Delivery surfaces

Surface	What it provides	Status
CLI (swarn)	run / solve / team / serve / mcp-serve / playbook / sessions — scriptable, CI-friendly	Shipped
Interactive REPL	Conversational agent with the full 40+ tool set	Shipped
Web dashboard	Live step feed (WebSocket), session browser, search-run tree viewer, playbook view	Shipped
MCP server	swarn_submit_task / swarn_task_status / swarn_get_messages / swarn_list_tasks for any MCP client	Shipped
IDE extension	VS Code / Cursor sidebar (HeyNeo parity)	Roadmap
Managed cloud	Zero-setup hosted runs (HeyNeo parity)	Roadmap

3. Use Cases

The platform serves the same seven workload families HeyNeo markets, executed through one uniform workflow (Section 4).

Category	Representative tasks	How Swarn executes them
Model building & AutoML	Tabular classification/regression, time series, anomaly detection, churn/fraud models	Tree search generates competing pipelines (feature engineering x model families x HPO), measures each on held-out splits, returns the best
LLM evaluation & benchmarking	Model comparisons, eval harnesses, LLM-as-judge suites, prompt A/B tests, regression gates	ReAct agent builds eval scripts and datasets; search optimizes prompts against measurable scores
Fine-tuning	LoRA/QLoRA instruction tuning, domain adaptation of open-weight models, quantized exports	Built-in PEFT pipeline (transformers + peft + datasets); GPU sandbox; checkpoint comparison in reports
RAG systems	Repo-RAG, multimodal RAG (PDF/OCR/audio), chunking & retrieval tuning, grounded QA	ChromaDB + sentence-transformers indexing tools; search loop tunes chunking/thresholds against retrieval metrics
Agent & pipeline development	Multi-step agent workflows, data pipelines, deployment packaging	Multi-agent orchestrator (planner-coder-reviewer-tester) plus deployment tools (FastAPI app, Dockerfile, ONNX/pickle export)
Debugging & optimization	Fix broken training pipelines, hunt data leakage, speed up inference	Self-correction policy classifies errors and injects hints; doom-loop guard prevents wasted cycles; static gate catches broken code pre-execution
Continuous engineering	Nightly retrains, scheduled evals, regression watchdogs via MCP/CLI in CI	Deterministic CLI exit codes; resumable runs; MCP task lifecycle for orchestration from other agents

Concrete example

```
swarn solve "Predict customer churn from ./data/churn.csv. Metric: AUC." -d ./data -s 24 --workers 4 --token-budget 500000
```

One command: the platform previews the data, injects lessons from past runs, explores 24 candidate solutions four-at-a-time inside its sandbox, and returns **best_solution.py**, **report.md** (solution tree + metric history + failure analysis), and an updated playbook.

4. End-to-End Workflow

4.1 The user's 4-step workflow

- **Describe the task.** Natural language, including the target and metric. No prompt engineering.
- **Point at context.** A data directory (auto-previewed: file tree, dtypes, CSV heads, JSON keys), optional evaluation notes. The model is fixed platform-wide: the deployed endpoint.
- **Let it run.** The engine iterates within explicit budgets (steps, wall-clock, tokens). Runs are crash-safe and resumable; long runs can be watched live on the dashboard or polled over MCP.
- **Review and steer.** Inspect the solution tree and report, resume with more steps, tighten the task, or promote `best_solution.py` to production.

4.2 Inside one search run

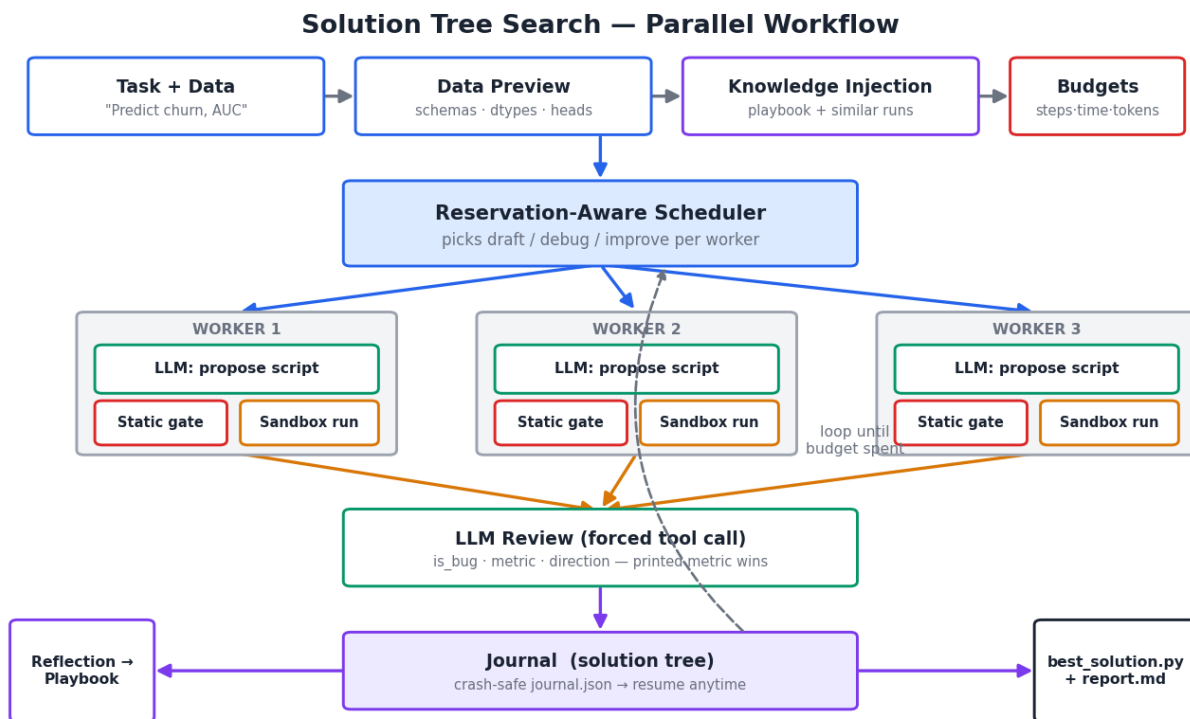


Figure 1 — Parallel solution tree search: propose → gate → execute → review → journal.

Scheduling. Each worker asks the policy for an action. The policy drafts until `num_drafts` independent roots exist (in-flight drafts count toward the quota), then debugs buggy leaves (cheapest-expected-fix first, capped at `max_debug_depth` consecutive attempts) with probability `debug_prob`, otherwise improves a top-k good node (epsilon-greedy: 70% best node, 30% a runner-up — preserving the diversity the drafts paid for). Reservations guarantee two workers never touch the same leaf.

Static gate. Before any sandbox execution, the candidate script is AST-parsed. Syntax errors, missing metric prints, and guaranteed hangs (`input()`) are scored buggy immediately — the debug branch fixes them one full sandbox round-trip cheaper. The gate only rejects certain failures; runtime uncertainty always executes.

Execution. Docker when available (memory/CPU caps, bind-mounted workspace, runaway-process kill on timeout) or a cross-platform subprocess backend (hard wall-clock timeouts, head+tail output truncation). Per-node timeouts shrink automatically to fit the remaining wall budget.

Review. A second, low-temperature LLM call is forced (tool call) to return `{is_bug, summary, metric, lower_is_better}`. The script's own printed 'Final Validation Metric' line overrides the reviewer on disagreement — regex plus reviewer, not vibes.

Journal. Every node (code, output, verdict, metric, lineage) is appended to a tree persisted after every step. This powers crash-safe resume (`--resume <run_id>`), the report, and the dashboard tree view.

4.3 The self-improvement loop

Self-Improvement Engine — Every Run Makes the Platform Smarter

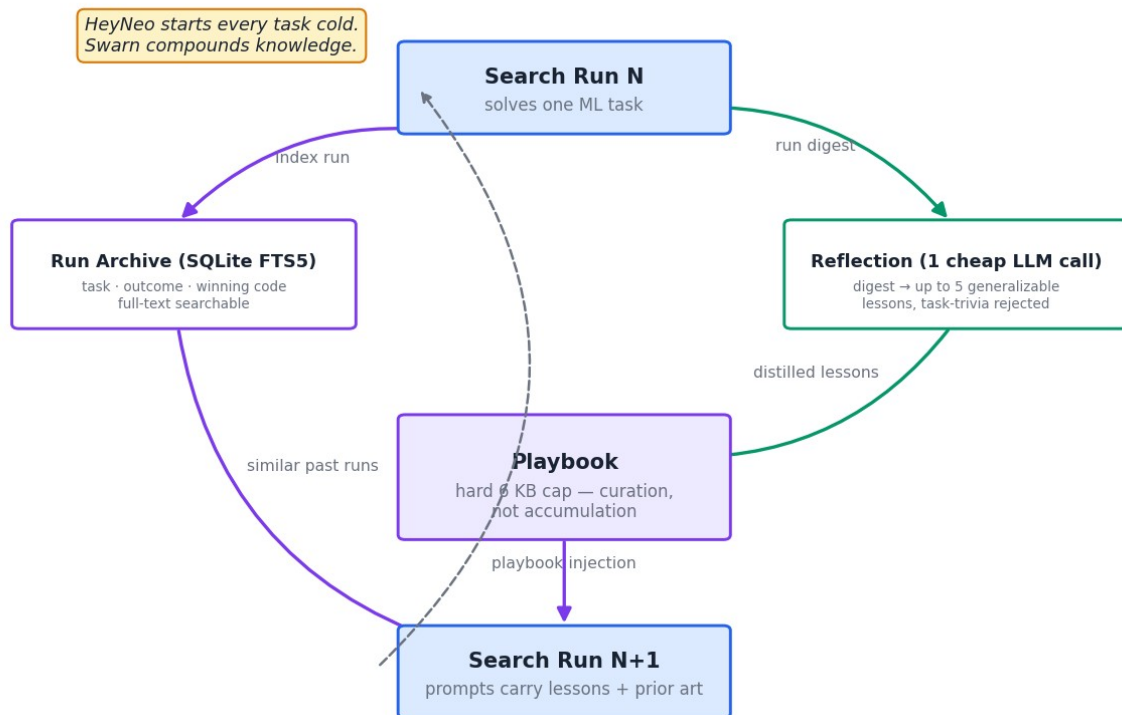


Figure 2 — Reflection distills lessons; the archive supplies prior art; future prompts carry both.

After a run completes, one cheap reflection call reviews the run digest and extracts up to five generalizable lessons — task-specific trivia is explicitly rejected by the prompt. Lessons merge into a playbook with a hard 6 KB cap: when full, the oldest lessons are dropped, forcing curation over accumulation (a design borrowed from NousResearch's hermes-agent). Every run is also indexed (task, outcome, winning code) in a SQLite FTS5 archive. At the start of any new run, the playbook plus the three most similar archived runs are injected into every prompt.

4.4 The ReAct agent workflow (non-search tasks)

For open-ended engineering tasks (fix this pipeline, build an eval harness, index this repo), the ReAct loop drives 40+ tools: data loading/validation, feature engineering, model training with Optuna HPO, evaluation with plots, deployment packaging, fine-tuning, RAG indexing/search, MCP client tools, and `solve_ml_task` itself. Three protective layers run on every tool result: guardrail scan (prompt-injection patterns), self-correction (error

classification + hints, abort after 3 consecutive failures), and the V3 doom-loop detector (same-call-same-result repetition triggers a corrective note; result-hashing means polling never false-positives). Conversations are auto-compacted beyond a size budget so long runs never blow the context window.

5. Feature Catalog

5.1 Search engine (agent/search/)

Feature	Detail
Parallel tree search	N workers pipeline propose/execute/review; reservation-aware policy; dense, unique journal steps under concurrency
Draft / debug / improve policy	Configurable num_drafts, debug_prob, max_debug_depth, improve_topk (epsilon-greedy)
Static gate	AST checks pre-execution: syntax, metric-print presence, interactive-input hangs
Budgets	steps, wall-clock seconds, LLM tokens; graceful drain of in-flight work on exhaustion
Checkpoint / resume	journal.json saved every step; --resume <run_id> adds more nodes to a finished or killed run
Trust-but-verify review	Forced-tool LLM review + printed-metric override + exit-code checks
Data preview	File tree, CSV dtypes/heads, JSON keys injected into prompts so code targets real columns
Reports	ASCII solution tree, metric history, failure analysis, token usage per run

5.2 Knowledge & self-improvement (agent/knowledge.py)

Feature	Detail
Bounded playbook	6 KB hard cap, dedup, oldest-dropped-first; inspect/clear via 'swarn playbook'
Post-run reflection	1 low-temp LLM call, forced tool, max 5 generalizable lessons; never fatal to the run
Run archive	SQLite FT55: task, summary, best metric, winning code; similarity search at run start
Prompt injection	Playbook + top-3 similar runs added to every draft/improve/debug prompt

5.3 LLM layer (agent/llm/)

Feature	Detail
Single deployed endpoint	One OpenAI-compatible /v1 endpoint serves every call — configured in agent/llm/router.py (TEST: Qwen 3.5 9B on Modal; PRODUCTION: swap the DEPLOYED_* constants or SWARN_DEPLOYED_* env vars)
Providers	The deployed model (served OpenAI-compatible, e.g. vLLM), plus a scripted mock for offline tests — BYO-LLM provider presets removed
Hard routing	Every model spec (CLI flags, env vars) hard-routes to the deployed endpoint; only mock:* passes through. SWARN_CODE_MODEL / SWARN_FEEDBACK_MODEL remain as display labels
Resilience & accounting	Exponential-backoff retries, normalized usage tracking feeding the token budget

5.4 Execution, safety, observability

Feature	Detail
Docker backend	Persistent container, bind-mounted workspace, mem/CPU caps, timeout kills the runaway container
Subprocess backend	Windows-safe (sys.executable), hard timeouts, structured ExecResult with exit code and timing
Guardrails	Prompt-injection scan on tool results + injection-benchmark harness
Self-correction	Error detection/classification, diagnostic hints, consecutive-error abort
Doom-loop guard	Result-hash-aware repetition detection with corrective prompt injection
Context compaction	Deterministic head+tail truncation of old tool results past a char budget
OpenTelemetry	Optional spans around every LLM call and tool call (OTLP exportable)
Sessions	Every ReAct run logged as typed steps; browsable via CLI and dashboard

5.5 ML pipeline tools (ReAct toolset)

Area	Capabilities
Data	CSV/Excel/Parquet/SQL/S3/GCS loaders, pandera schema validation, z-score outlier checks, missing-data handling
Features	sklearn ColumnTransformer pipelines, leakage-safe fitted transformers
Training	Linear/logistic, RandomForest, XGBoost, LightGBM, PyTorch MLP; task auto-detection; Optuna HPO
Evaluation	Classification/regression metrics, confusion matrices, ROC/PR curves, residual plots
Deployment	joblib/ONNX export, generated FastAPI serving app + requirements + Dockerfile
Fine-tuning	PEFT LoRA/QLoRA on open-weight models (transformers/datasets/accelerate; QLoRA on CUDA)
RAG	AST-chunked repo indexing, multimodal ingestion (PDF text/tables, OCR, Whisper audio) into ChromaDB
Multi-agent	Planner → Coder → Reviewer → Tester pipeline with role-restricted toolsets

6. High-Level Architecture

Swarn — High-Level Architecture

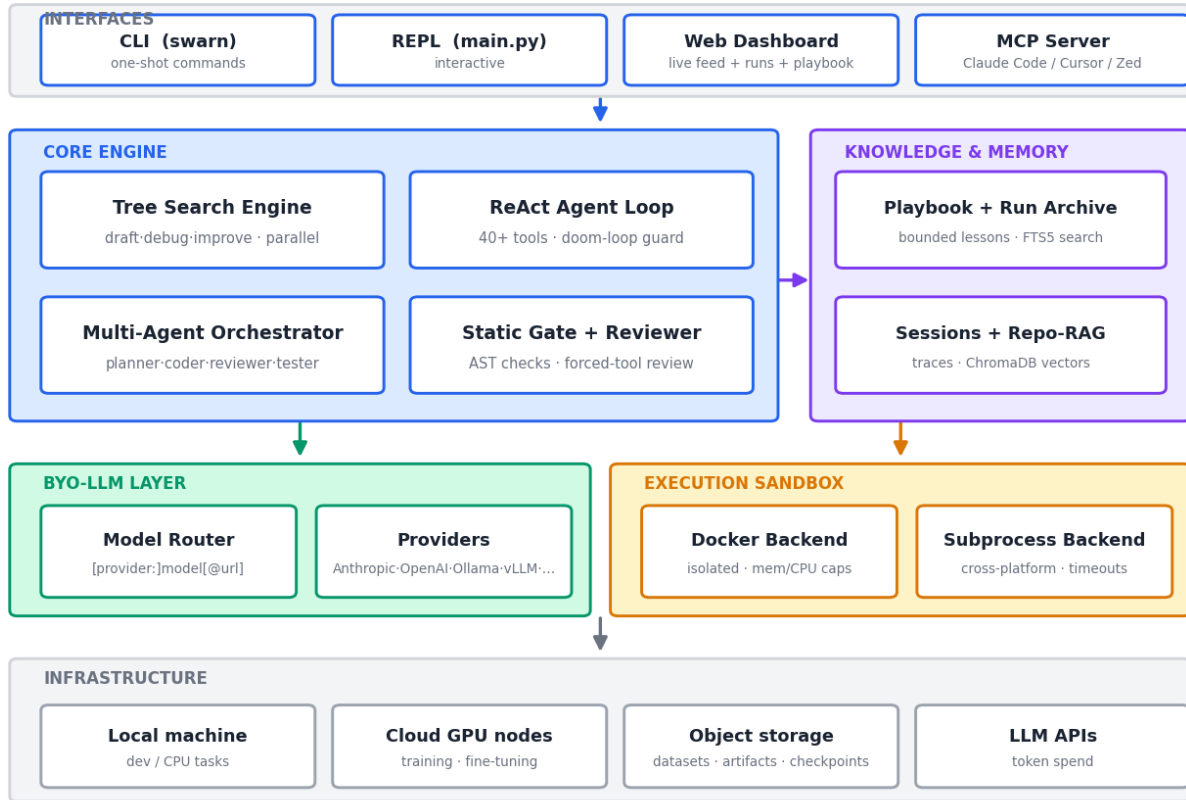


Figure 3 — Five layers: interfaces, core engine, knowledge, deployed model + execution, infrastructure.

Interfaces. Four thin entry points (CLI, REPL, dashboard, MCP server) share one engine. Nothing user-facing holds state of its own; everything reads the same journals, sessions, and knowledge stores on disk.

Core engine. Two complementary brains: the tree-search engine for measurable ML tasks (optimizes a metric under budget) and the ReAct loop for open-ended engineering (drives tools). The orchestrator composes role-restricted ReAct instances into a review pipeline. The static gate and forced-tool reviewer keep both honest.

Knowledge & memory. The self-improvement stores (playbook + FTS5 archive), per-run journals, ReAct session traces, and the ChromaDB vector store for repo/multimodal RAG.

LLM + execution. One deployed model behind one endpoint (agent/llm/router.py — the single place to swap TEST Qwen for the production model); any code inside one sandbox interface. Both are swappable without touching engine code — the mock LLM client and the subprocess backend make the entire engine testable offline.

Infrastructure. Local-first by default; scales to CPU/GPU cloud nodes without architectural change (Section 8).

7. Low-Level Architecture

Swarn — Low-Level Module Architecture (agent/ package)

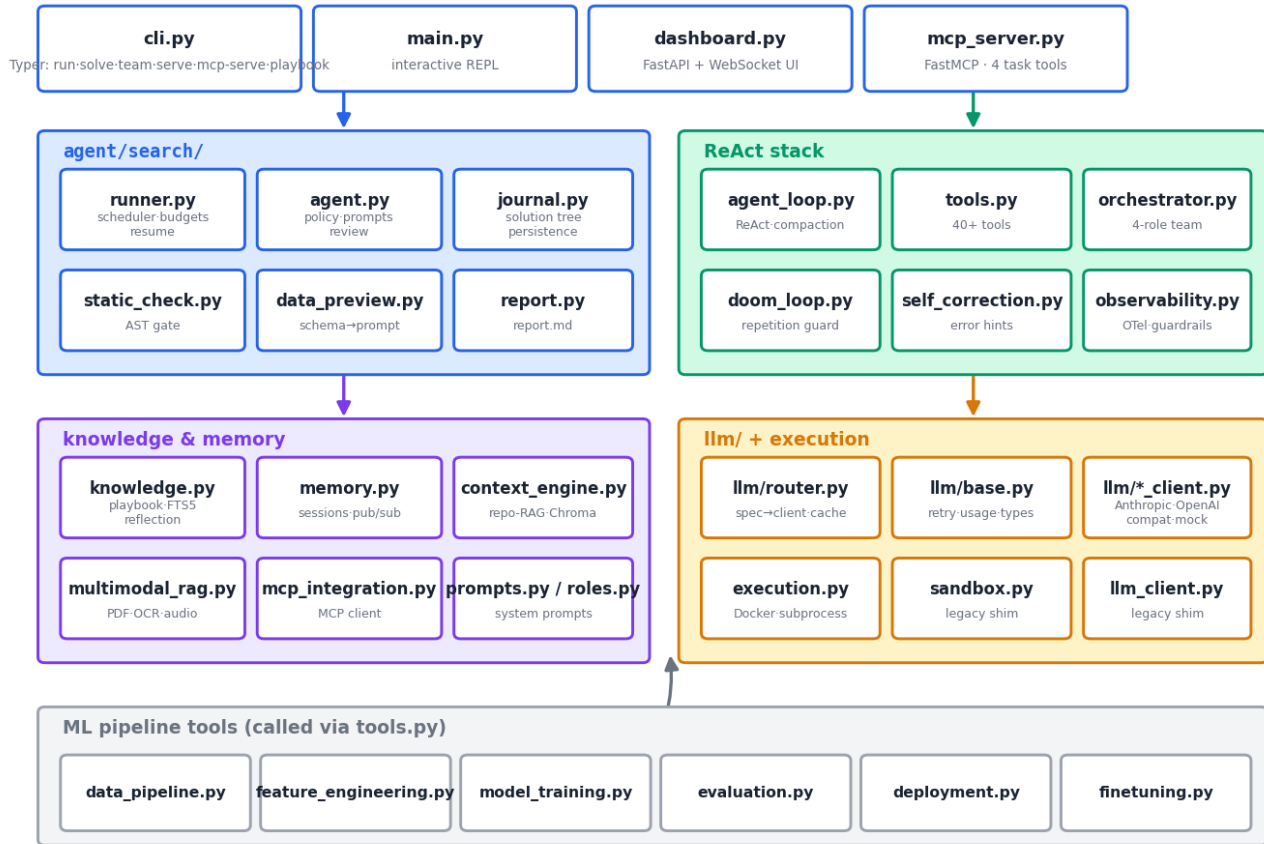


Figure 4 — Module map of the agent/ package (~10,500 LOC Python).

7.1 Key design decisions

- **Threads over async.** The parallel scheduler uses a ThreadPoolExecutor: LLM calls and sandbox executions are I/O-bound, workers share no state (the scheduler owns the journal under a lock), and synchronous provider SDKs stay simple.
- **The journal is the source of truth.** Policy decisions, resume, reports, and the dashboard all derive from the persisted tree — there is no hidden in-memory state to lose.
- **Everything degrades gracefully.** No Docker → subprocess. No knowledge dir → empty context. Reflection fails → run still succeeds. Review call fails → printed metric wins. A worker exception costs one node, never the run.
- **Offline testability as an architectural property.** The scripted mock provider + subprocess backend let 57 automated tests exercise the full loop — including parallel scheduling, resume, budgets, gating, and the complete self-improvement cycle — with zero network calls.

7.2 Data flow of one node (sequence)

#	Step	Component	Artifact
1	Policy picks draft/debug/improve (+ parent)	search/agent.py choose_action	reservation

#	Step	Component	Artifact
2	Prompt assembled: task + data preview + playbook + similar runs + journal memory	search/agent.py + knowledge.py	prompt
3	Code LLM generates plan + complete script	llm/router → deployed endpoint	Node(plan, code)
4	Static gate parses AST	search/static_check.py	gate verdict
5	Sandbox executes with fitted timeout	execution.py	ExecResult
6	Feedback LLM reviews via forced tool; printed metric overrides	search/agent.py review	verdict + metric
7	Node appended, tree saved	search/journal.py	journal.json
8	On run end: reflect + archive + report	knowledge.py + report.py	playbook, runs.db, report.md

8. Infrastructure Requirements

The platform is local-first: everything runs on a laptop for tabular-scale work. Competing with HeyNeo's 'runs for days on your GPU cloud' story requires the tiers below.

Infrastructure Requirements & Deployment Topology

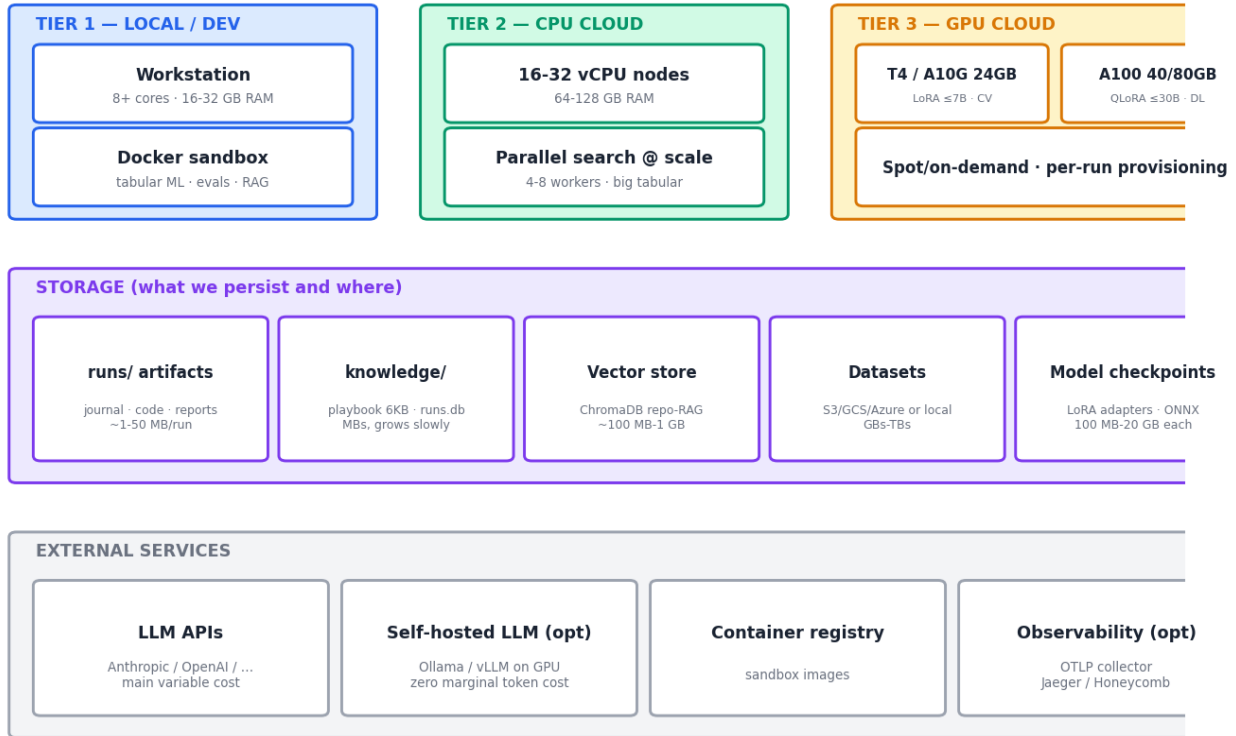


Figure 5 — Compute tiers, storage inventory, external services.

8.1 Compute

Tier	Hardware	Workloads	Notes
Local / dev	8+ cores, 16-32 GB RAM, Docker Desktop	Tabular ML, evals, RAG indexing, prompt optimization	Default; subprocess fallback works without Docker
CPU cloud	16-32 vCPU, 64-128 GB RAM (e.g. c7i.4xlarge-8xlarge)	Parallel search (4-8 workers) on large tabular data; batch evals; CI	One node per concurrent run; stateless — journal on persistent volume
GPU - entry	NVIDIA T4/A10G (16-24 GB VRAM), 8 vCPU, 32 GB RAM	LoRA fine-tunes up to ~7B, CV training, embedding jobs	Spot instances fine — runs are resumable
GPU - heavy	A100 40/80 GB (or H100), 12+ vCPU, 128 GB RAM	QLoRA up to ~30B, deep learning experiments, large batch inference	On-demand for long runs; VRAM rule of thumb: bf16 ~2 bytes/param + ~20% optimizer overhead
Deployed LLM serving (the only LLM path)	1-2x A100/H100 running vLLM	Zero-marginal-cost code generation for the search loop	TEST: Qwen 3.5 9B on Modal (autoscaling, scale-to-zero); PRODUCTION: swap endpoint via SWARN_DEPLOYED_*

8.2 Storage

Store	Contents	Size envelope	Medium
runs/	Per-run journal.json, workspace, best_solution.py, report.md	1-50 MB per run; grows linearly	Local disk / persistent volume; archive old runs to object storage
knowledge/	playbook.md (6 KB cap) + runs.db (FTS5 archive)	MBs; grows slowly by design	Local disk; back up — this is the platform's accumulated learning
Vector store	ChromaDB collections (repo-RAG, multimodal)	100 MB - 1 GB per indexed corpus	Local disk
Datasets	User training data (copied or symlinked into run workspaces)	GBs - TBs	S3 / GCS / Azure Blob; loaders already support cloud URIs
Model checkpoints	LoRA adapters (100-500 MB), full fine-tunes (up to ~20 GB), ONNX/pickle exports	GBs per fine-tune	Object storage with lifecycle rules
Sessions & traces	ReAct session JSON + summaries	KBs - MBs per session	Local disk

8.3 External services & variable costs

Service	Purpose	Cost driver
Deployed model serving (self-hosted GPU)	Code generation + review calls	GPU-hours instead of per-token API fees — predictable COGS. A 20-step search run: ~40+ LLM calls; --token-budget still caps each run; serverless autoscaling (e.g. Modal) scales to zero between runs
GPU cloud (AWS/GCP/Lambda/vast.ai)	Fine-tuning + DL experiments	Hourly; spot-friendly because runs resume
Object storage	Datasets, checkpoints, archived runs	Cents/GB-month
Container registry	Sandbox images (python:3.11-slim + ML stack)	Negligible
OTLP collector (optional)	Production tracing (Jaeger/Honeycomb)	Per-span pricing or self-hosted

8.4 What we explicitly do NOT need

- No third-party LLM APIs — every call goes to our own deployed open-weight model (TEST: Qwen 3.5 9B on Modal; the production endpoint is one config swap).
- No Kubernetes for v1 — one process per run; a queue + VM pool suffices for early scale.
- No customer-data lake — local-first design keeps user data on user infrastructure (a selling point HeyNeo also markets).

9. Competitive Positioning vs HeyNeo

Dimension	HeyNeo (documented)	Swarn (this platform)
Search strategy	Iterative experimentation, one node at a time ('100s of experiments')	Same tree policy plus parallel workers — N drafts for one draft's wall time
Cross-task learning	None documented — every task starts cold	Playbook + FTS5 run archive injected into every run; compounds with use
Run continuity	'Runs for days'; no resume documented	Crash-safe journal + --resume; spot-instance friendly
Cost control	Opaque credit system (Lite/Pro modes undocumented)	Explicit token/time/step budgets per run; transparent usage in reports
LLM strategy	Anthropic, OpenAI, OpenRouter — per-token API fees	One self-hosted deployed open-weight model — full-stack control, predictable GPU COGS, zero per-token fees; archived winning runs become fine-tuning data for OUR model
Failed-code cost	Executes everything	Static gate scores guaranteed failures without a sandbox run
Loop protection	Auto Mode self-check (coarse)	Doom-loop detector (result-hash aware) + context compaction + self-correction
IDE / MCP	VS Code + Cursor extension; neo-mcp (4 tools)	MCP server shipped (same 4-tool lifecycle); IDE extension on roadmap
Managed cloud	Yes (50 MB upload cap, 7-day session expiry, CPU default)	Roadmap
Benchmark claim	#1 MLE-Bench 34.2% (Aug 2025)	To be validated — engine implements the AIDE-family approach that leads that benchmark, plus parallelism and learning on top

10. Testing & Quality

57 automated tests (up from 31 pre-V3) run offline in seconds — no API keys, no network — via a scripted mock LLM provider and the subprocess backend.

Suite	Covers
test_llm_layer	Deployed-endpoint routing (spec-ignore + mock passthrough), message/tool translation, retry semantics
test_execution	Subprocess execution, timeouts, truncation, exit codes, working dirs
test_journal	Tree operations, best-node selection, persistence round-trips, prompt memory
test_search_agent	Policy transitions, top-k improve bounds, reservations, review parsing, printed-metric override
test_static_check	Every gate rule, plus the no-false-veto guarantee
test_knowledge	Playbook curation/caps/dedup, FTS5 archive search, reflection (incl. junk-input safety)
test_doom_loop	Repetition shapes, polling false-positive immunity, compaction budgets

Suite	Covers
test_e2e_search	Full sequential run on synthetic data: broken draft → working solution → artifacts
test_parallel_resume	Parallel journal integrity, resume continuation, gate-skips-execution, token-budget stop

Additionally verified end-to-end: the complete self-improvement cycle (run 1 learns a lesson → run 2's prompts carry it plus similar-run prior art), CLI surface, and MCP server tool registration.

11. Roadmap to Market

Phase	Scope	Est. effort
1. Benchmark validation	Run MLE-Bench-style suite with real models; tune policy constants; publish results — the marketing centerpiece	2-3 weeks + GPU/API spend
2. IDE extension	VS Code/Cursor sidebar wrapping the MCP server: task submit, live tree view, playbook browser	3-4 weeks
3. Managed cloud beta	Auth + run queue + VM pool + artifact download; per-seat billing with transparent token metering	4-6 weeks
4. Hardening	Multi-user isolation, secrets vault, richer guardrails, SSO/audit logs for enterprise	ongoing

Appendix A — CLI Reference

```

swarn run "task"           one-shot ReAct agent (full toolset)
swarn solve "task" -d DATA -s N --workers W --token-budget T  tree search
swarn solve "task" --resume RUN_ID -s N                        continue a run
swarn team "task"         planner->coder->reviewer->tester pipeline
swarn playbook [--clear]  view / reset learned lessons
swarn mcp-serve            MCP server (Claude Code, Cursor, Zed, ...)
swarn serve [-p PORT]     web dashboard
swarn sessions / recall / index / guardrail-benchmark         utilities

```

Appendix B — Environment Variables

Variable	Purpose	Default
SWARN_DEPLOYED_MODEL / _BASE_URL / _API_KEY	Deployed model endpoint — overrides the defaults in agent/llm/router.py	qwen3.5-9b @ Modal (TEST)
SWARN_CODE_MODEL / SWARN_FEEDBACK_MODEL	Display labels only; mock:* runs the engine offline (tests)	—
SWARN_SANDBOX	docker subprocess	auto-detect

Variable	Purpose	Default
SWARN_EXEC_TIMEOUT / SWARN_SANDBOX_IMAGE	Per-call timeout (s) / container image	300 / python:3.11-slim
SWARN_SEARCH_WORKERS	Parallel solution nodes	1
SWARN_KNOWLEDGE_DIR	Playbook + archive location	<repo>/knowledge
SWARN_MAX_ITERATIONS	ReAct iteration cap	30
SWARN_CONTEXT_CHAR_BUDGET	Compaction threshold (chars)	400000